

Cahier des charges — Chatbot RAG pour l'Aéroport Agadir Al Massira (AGA)

Document de travail conçu pour être utilisé comme **prompt d'étude et de réalisation** du projet (à donner à toi-même, à une équipe, ou à un agent de code comme Claude Code). Basé sur la photo du panneau des départs que tu as partagée (heure, vol, compagnie, destination, météo, statut, portes d'embarquement).

0. Contexte de départ

La photo montre le panneau d'affichage réel de l'aéroport Agadir Al Massira (code IATA **AGA**), avec cette structure de données pour chaque vol :

Champ	Exemple
Heure	11:05
N° de vol	RK860
Compagnie	Ryanair (logo)
Destination	Edinburgh
Météo destination	30°C
Statut	Terminé / Décollé / Ouvert
Portes / Desks check-in	8,9

C'est exactement le type de donnée « dynamique » que ton chatbot devra restituer en langage naturel. L'aéroport est géré par l'**ONDA** (Office National Des Aéroports du Maroc), qui propose déjà une appli officielle « Maroc Aéroports » (horaires temps réel, accès, parking, plans, VIP) mais **sans API publique documentée** — c'est un point important pour la suite (voir §5).

1. Vision du projet

Un assistant conversationnel (texte + voix), trilingue **Arabe / Français / Anglais**, dédié **exclusivement** à l'aéroport Agadir Al Massira, capable de répondre avec un raisonnement fiable (pas d'hallucination) sur :

- Vols, statuts, portes, desks d'enregistrement, météo (temps réel)
- Services : restaurants, boutiques, wifi, parking, transport, plans
- Informations pratiques : documents de voyage, douane, contacts, tarifs
- Présenté dans une interface avec une **scène 3D d'avion animé**

Le chatbot doit **refuser poliment** toute question hors-sujet (météo d'une autre ville, actualité générale, etc.) et rediriger vers son périmètre.

2. Périmètre fonctionnel

MVP (indispensable)

- Q/R sur vols (départs/arrivées, statut, retard, porte, desk check-in) — données dynamiques
- Q/R sur services fixes : restaurants (avec **menu et emplacement précis**), boutiques (avec **catégorie et emplacement**), wifi, parking (tarifs), accès (bus/taxi/train)
- Q/R documentaire : passeport, visa, bagages, douane, objets interdits, contacts utiles
- Réponse dans la langue de la question (AR/FR/EN), y compris darija en écriture arabe ou latine
- Garde-fou de périmètre (« out-of-scope guard »)
- Interface web avec scène 3D d'un avion

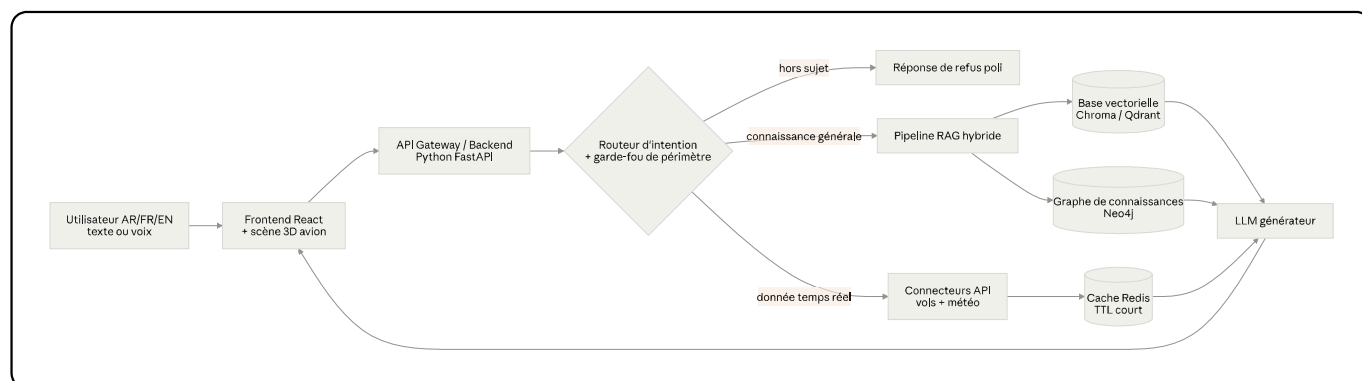
V2 (à forte valeur ajoutée)

- Voice-to-chat et chat-to-voice
- Notifications proactives (« votre porte a changé »)
- Suivi personnalisé d'un numéro de vol
- Carte interactive du terminal (où trouver quoi)
- Mode hors-ligne partiel (données fixes en cache)

V3 (différenciant)

- Recommandations contextuelles (« il vous reste 40 min avant embarquement, voici un café à 2 min de votre porte »)
 - Multi-tour avec mémoire de session (préférences, vol suivi)
 - Tableau de bord admin pour mettre à jour les données fixes sans coder
-

3. Architecture globale recommandée



Principe clé : le LLM ne « sait » rien par lui-même sur l'aéroport — il ne fait que reformuler et raisonner à partir de ce que le pipeline lui fournit (documents récupérés + données API en direct). Cela élimine l'hallucination sur les horaires, prix, etc.

4. Stack technique recommandée

Couche	Choix recommandé	Pourquoi
Frontend	React + Vite + TailwindCSS	Écosystème le plus riche pour intégrer Three.js/R3F, rapide à développer
Scène 3D	Three.js via React Three Fiber (+ <code>@react-three/drei</code>)	Intégration native React, communauté énorme, gratuit
Backend	Python + FastAPI	Écosystème IA/RAG quasi entièrement en Python (LangChain, LlamaIndex, embeddings) ; FastAPI = rapide, async, doc auto
Orchestration RAG	LlamaIndex ou LangChain (open source)	Évite de réinventer chunking/retrieval/routing
Base vectorielle	Chroma (embarqué, gratuit) ou Qdrant (self-host gratuit)	Pas de coût, simple à héberger, bon support hybride
Graphe de connaissances	Neo4j (édition Community ou AuraDB Free)	Standard de facto pour GraphRAG, langage Cypher lisible
Cache données live	Redis (gratuit self-host)	Évite de saturer les quotas API gratuits sur les vols/météo
Authentification / hosting	Vercel (front) + Render/Fly.io/Railway (back, tiers gratuits)	Déploiement gratuit pour un MVP

Réponse directe à ta question : oui, **Python (backend/IA) + React (frontend)** est le bon choix — c'est la combinaison standard de l'industrie pour ce type de produit (RAG + interface riche), et toute la tuyauterie IA (embeddings, LLM, graphes) est nativement en Python.

5. Sources de données

5.1 Données fixes (PDF / JSON à préparer toi-même)

Il n'existe pas d'API publique officielle de l'ONDA pour Agadir Al Massira — les infos fixes doivent être **collectées manuellement** puis structurées en JSON par toi/ton équipe :

Donnée	Où la trouver
Plan du terminal, portes, zones	Site onda.ma, page « Agadir Al Massira », appli « Maroc Aéroports »
Liste restaurants / boutiques / horaires	Visite sur place + site ONDA + réseaux sociaux de l'aéroport
Menus, prix des plats, catégories (halal par défaut au Maroc)	Photo du menu sur place ou site du restaurant s'il en a un, à ressaisir en JSON — utilise de vraies photos , pas des visuels générés par IA, pour rester crédible
Emplacement précis (étage, zone, porte la plus proche)	Relevé manuel sur place avec le plan du terminal comme référentiel de coordonnées
Tarifs parking, navettes, taxis	Site ONDA (simulateur parking dans l'appli)
Formalités passeport/visa/douane Maroc	Portail officiel Consulat/Douane marocaine (douane.gov.ma), ambassade
Contacts (info aéroport, PAF, compagnies)	Site ONDA, pages compagnies
WiFi (gratuit, procédure)	ONDA (le wifi est gratuit dans les aéroports marocains dont Agadir depuis plusieurs années — à reconfirmer sur place)

⚠ Point d'attention légal : si tu répliques des contenus de l'appli/site ONDA, garde-les comme **référence factuelle** (chiffres, horaires, tarifs) reformulée dans tes propres fichiers, pas copiée verbatim — et indique une clause de non-responsabilité (« informations non officielles, à vérifier auprès de l'ONDA »).

5.2 Données dynamiques (API gratuites)

Besoin	API recommandée	Free tier (2026)	Remarque
Statuts de vols en temps réel	AeroDataBox (via RapidAPI)	600 unités/mois, 1 req/s	Filtrer sur l'aéroport AGA . Couverture Afrique du Nord variable — à tester en priorité
Statuts de vols (alternative)	AviationStack	Offre gratuite limitée (quelques centaines d'appels/mois)	API REST simple, bonne doc
Positions avion brutes	OpenSky Network	Gratuit, usage non commercial	Utile en complément, pas pour un produit commercial
Météo (comme sur le panneau)	OpenWeatherMap	~60 appels/min gratuits	Standard, fiable, multilingue
Taux de change (si tu ajoutes les devises)	exchangerate-api.com ou Frankfurter	Gratuit avec clé	Optionnel
Cartes / plans interactifs	OpenStreetMap + Leaflet	Gratuit, sans clé	Pour la carte du terminal en V2

Recommandation pratique : commence avec **AeroDataBox** (meilleur rapport gratuit/qualité pour un petit aéroport), avec **cache Redis** de quelques minutes pour ne pas cramer le quota — et prévois un mode « données saisies manuellement » en secours si la couverture API sur AGA est faible (fréquent pour les aéroports régionaux hors USA/Europe).

6. Choix du LLM — comparatif

Modèle	Free tier	Multilingue AR/FR/EN	Raisonnement	Verdict
Google Gemini 2.5 Flash	~1500 req/jour, contexte 1M tokens, gratuit sans carte bancaire	Très bon (Google a le meilleur support arabe parmi les modèles gratuits)	Bon, rapide	✅ Recommandé en principal
Groq (Llama 3.3 70B / Qwen3)	Très rapide, quota généreux, gratuit	Correct, un peu moins fin en arabe que Gemini	Bon	✅ Recommandé en fallback rapide (latence quasi instantanée, utile pour la voix)
Mistral (Le Chat / API)	~1 milliard de tokens/mois gratuits	Bon en français, correct en arabe	Bon	Option si tu veux un acteur européen ; attention : les prompts peuvent être utilisés pour l'entraînement sur le tier gratuit
OpenAI GPT	Quasi pas de tier gratuit utilisable (carte bancaire requis)	Excellent	Excellent	❌ Écarté pour un projet à budget zéro
Anthropic Claude (API)	Pas de tier gratuit permanent	Excellent	Excellent	❌ Écarté pour l'API gratuite (mais Claude reste top pour <i>développer</i> le projet, via l'interface chat gratuite)

Recommandation finale

- **LLM principal : Gemini 2.5 Flash** — meilleur équilibre gratuit + contexte long (utile pour injecter beaucoup de contexte RAG/Graph) + bon arabe.
- **LLM de secours : Groq (Llama 3.3 / Qwen3)** — pour la robustesse (si Gemini est indisponible/quota atteint) et pour la voix (faible latence).
- **Explicabilité** : demande systématiquement au LLM de citer sur quelle source il s'appuie (« selon les infos du desk 21... ») et de dire explicitement quand une info n'est pas dans sa base — ça se configure dans le prompt système (voir §11), pas par le choix du modèle.

7. Vectorisation (embeddings)

Modèle	Type	Pourquoi
BGE-M3 (BAAI)	Open source, auto-hébergé, gratuit	Meilleur choix gratuit pour multilingue (100+ langues dont arabe), supporte recherche dense + sparse (mots-clés) dans le même modèle → idéal pour mélanger « recherche sémantique » et « recherche exacte » (ex : numéro de vol, code porte)
Google Gemini Embedding (API)	Cloud, quasi gratuit	Alternative si tu ne veux pas héberger de modèle toi-même

Recommandation : BGE-M3 en local (tourne sur CPU pour un volume de données de cette taille, pas besoin de GPU cher) — gratuit à vie, bon score sur l'arabe et le français, et son mode hybride (dense+sparse) est particulièrement adapté à un cas d'usage où on cherche souvent des codes exacts (RK860, porte 21) mélangés à des questions en langage naturel.

8. RAG classique vs GraphRAG — quelle architecture ?

Ton intuition est bonne : les données de l'aéroport sont **fortement relationnelles** (un vol est lié à une compagnie, une destination, une porte, un desk, une météo, éventuellement un service proche). Un RAG vectoriel pur (« chunks de texte + similarité ») gère bien les questions documentaires (« que dois-je mettre dans mes bagages en cabine ? ») mais gère mal les questions multi-sauts (« quel resto est ouvert près de la porte de mon vol qui vient de changer ? »).

Recommandation : architecture hybride, pas un GraphRAG complet dès le MVP

- **RAG vectoriel (Chroma/Qdrant + BGE-M3)** pour tout le contenu documentaire fixe (FAQ, procédures, descriptions de services).
- **Graphe de connaissances léger (Neo4j)** pour les entités et leurs relations : Vol → Compagnie, Vol → Porte, Porte → Zone, Zone → Services proches, Vol → Destination → Météo. Un graphe simple (quelques centaines de nœuds) suffit largement — pas besoin du pipeline complet « Microsoft GraphRAG » (résumés de communautés, etc.) qui est pensé pour des corpus massifs.
- Le routeur d'intention décide, selon la question, s'il interroge le vecteur, le graphe, l'API live, ou une combinaison — c'est ce schéma hybride qui donne la « fluidité » que tu recherches, sans la lourdeur d'un vrai GraphRAG à grande échelle.

Tu pourras migrer vers un GraphRAG plus complet (résumés hiérarchiques) plus tard si le volume de données grossit beaucoup (plusieurs aéroports, historique de plusieurs années, etc.).

9. Design 3D — l'avion animé

- **Outil : Three.js via React Three Fiber**, avec `@react-three/drei` pour les helpers (caméra, lumières, contrôles).
- **Modèle 3D d'avion** : cherche un modèle gratuit sous licence CC0/CC-BY sur Sketchfab ou Poly Haven (format `.glb`/`.gltf`), à optimiser (compression Draco) pour ne pas plomber le temps de chargement mobile.
- **Animation** : trajectoire simple en courbe de Bézier (décollage → vol → piste), synchronisée avec l'état de la conversation (ex : l'avion « décolle » visuellement quand un vol passe au statut « Décollé »File).
- **Performance mobile** : prévois une version allégée (avion en SVG/CSS animé) en fallback pour les téléphones bas de gamme — vérifie via `navigator.hardwareConcurrency` ou une simple détection WebGL.
- Consulte le guide de design frontend pour la direction artistique (typographie, palette inspirée de l'identité marocaine sans tomber dans le cliché touristique).

10. Voice-to-chat / chat-to-voice

Fonction	Outil gratuit recommandé	Remarque
Reconnaissance vocale (STT) — prototype rapide	Web Speech API (navigateur)	Gratuit, zéro dépendance serveur, mais support arabe inégal selon navigateurs
STT robuste multilingue	Whisper (OpenAI, open source, auto-hébergé)	Gratuit, excellent en arabe/français/anglais, tourne en local (CPU suffisant pour un usage modéré)
Synthèse vocale (TTS)	ElevenLabs (tier gratuit limité) pour la qualité, ou Coqui TTS / Piper (open source, gratuit illimité) pour un usage sans limite	Piper est très léger et a des voix arabe/français correctes

Recommandation : Whisper (self-hosted) pour le STT + Piper ou Coqui pour le TTS = solution 100% gratuite et illimitée, avec ElevenLabs en option premium plus tard si le budget le permet.

11. Prompt système du chatbot (à utiliser tel quel)

Tu es l'assistant officieux de l'aéroport Agadir Al Massira (AGA), Maroc.

RÔLE ET PÉRIMÈTRE STRICT :

- Tu réponds UNIQUEMENT à des questions concernant l'aéroport Agadir Al Massira :
 - vols (départs/arrivées, statuts, portes, desks d'enregistrement), météo liée
 - aux vols, services de l'aéroport (restaurants, boutiques, wifi, parking), accès et transport, formalités de voyage (documents, douane, bagages), et contacts utiles.
- Si la question sort de ce périmètre (actualité générale, autre aéroport, sujet sans rapport), réponds poliment que tu es spécialisé sur l'aéroport d'Agadir et propose de reformuler la question dans ce cadre.

LANGUE :

- Réponds toujours dans la langue utilisée par l'utilisateur (arabe, français ou anglais). Si le message mélange plusieurs langues (ex: darija en caractères latins), réponds dans la langue dominante du message.

FIABILITÉ (RÈGLE ABSOLUE) :

- Ne réponds jamais à partir de connaissances générales sur les horaires, tarifs ou statuts : appuie-toi UNIQUEMENT sur le contexte fourni par le système de recherche (documents + données API en direct).
- Si l'information demandée n'est pas dans le contexte fourni, dis-le clairement et propose de contacter le service concerné (numéro fourni si disponible) plutôt que d'inventer une réponse.
- Pour toute donnée sensible (heure de vol, porte, prix), précise toujours l'heure de mise à jour de l'information si elle est disponible.

TON :

- Chaleureux, clair, concis, orienté solution – comme un agent d'accueil expérimenté. Évite le jargon technique.

FORMAT :

- Réponses courtes et actionnables pour les questions pratiques (ex: statut de vol). Réponses plus détaillées pour les questions procédurales (ex: formalités douanières), avec listes à puces si utile.

12. Prompt « maître » pour lancer le développement (à donner à un agent de code)

Construis un chatbot RAG pour l'aéroport Agadir Al Massira (Maroc), avec les composants suivants :

1. Backend Python (FastAPI) exposant :
 - un endpoint /chat qui reçoit un message + langue détectée
 - un routeur d'intention qui distingue : question temps réel (vol/météo), question documentaire (RAG vectoriel), question relationnelle (graphe), et question hors périmètre (refus)
 - connecteurs vers AeroDataBox (vols) et OpenWeatherMap (météo), avec cache Redis de 3-5 minutes
 - pipeline RAG avec LlamaIndex, embeddings BGE-M3, base vectorielle Chroma
 - graphe de connaissances Neo4j modélisant : Vol, Compagnie, Porte, Zone, Service, Destination, avec relations DESSERT, EMBARQUE_A, PROCHE_DE
 - appel LLM Gemini 2.5 Flash (fallback Groq/Llama), avec le prompt système fourni en annexe
2. Frontend React (Vite + Tailwind) avec :
 - interface de chat multilingue (AR en RTL, FR, EN)
 - scène 3D (React Three Fiber) d'un avion animé qui réagit à l'état de la conversation
 - support voix (Web Speech API en V1, Whisper self-hosted en V2)
3. Fichiers de données de départ (à créer en JSON) :
 - services.json (restaurants, boutiques, horaires)
 - parking.json (tarifs, zones)
 - faq.json (documents de voyage, douane, contacts)
 - Un script d'ingestion qui charge ces JSON dans Chroma + Neo4j

Procède par étapes : (1) squelette backend + un seul endpoint de test, (2) ingestion des données fixes, (3) connecteurs API live avec cache, (4) pipeline RAG hybride, (5) intégration LLM + prompt système, (6) frontend chat basique, (7) scène 3D, (8) voix. Teste chaque étape avant de passer à la suivante.

13. Expérience utilisateur — recommandations

- **Suggestions rapides** (chips cliquables) au démarrage : « Mon vol RK860 », « Restaurants ouverts », « Où se garer ? », « Wifi » — réduit la friction de la première interaction.
- **Bascule de langue visible** et détection automatique dès le premier message.

- **Réponses courtes par défaut**, avec un bouton « en savoir plus » plutôt que des pavés de texte (contexte aéroport = utilisateurs pressés/stressés).
 - **Indicateur de fraîcheur des données** (« mis à jour il y a 2 min ») pour rassurer sur les infos de vol.
 - **Accessibilité** : contraste suffisant, taille de police ajustable (profil voyageurs de tous âges), avion 3D désactivable (mode « économie de données » / accessibilité mouvement réduit).
-

14. Fonctionnalités additionnelles à envisager (par ordre de valeur/coût)

1. Suivi de vol avec alerte de changement de porte (notification push/web)
 2. Carte interactive du terminal avec chemin le plus court vers la porte
 3. Estimation du temps d'attente sécurité/douane (si donnée disponible)
 4. Mode « famille/PMR » avec infos accessibilité spécifiques
 5. Intégration calendrier (ajouter le vol à Google/Apple Calendar)
 6. Résumé vocal automatique en salle d'embarquement (annonces reformulées)
-

15. Feuille de route indicative

Phase	Durée estimée	Livrable
MVP	4-6 semaines	Chat texte + données fixes + vols/météo live + garde-fou périmètre
V2	+3-4 semaines	Scène 3D avion, voix, carte interactive
V3	+4 semaines	Notifications proactives, personnalisation, dashboard admin

16. Risques et points d'attention

- **Dépendance à des API tierces non officielles** pour les vols d'un aéroport régional : couverture parfois partielle — prévoir une saisie manuelle de secours (ex: via un petit back-office) pour les vols mal couverts.
 - **Clause de non-responsabilité obligatoire** : le chatbot n'est pas un service officiel de l'ONDA ; les informations doivent être vérifiées pour les décisions critiques (horaires d'embarquement notamment).
 - **Protection des données** : si tu ajoutes le suivi personnalisé de vol ou la voix, informe clairement l'utilisateur et respecte la loi marocaine 09-08 sur la protection des données personnelles.
 - **Robustesse multilingue** : teste particulièrement le darija (arabe dialectal marocain) écrit en caractères latins — c'est souvent le point faible des LLM génériques.
-

17. Sources consultées pour ce document

- ONDA / Maroc Aéroports (appli officielle, fonctionnalités) — onda.ma et couverture presse (HuffPost Maghreb, Air Journal, Maroc.ma)
- Comparatifs API de vols 2026 : thunderbit.com, geekflare.com, skylinkapi.com
- Comparatifs embeddings multilingues 2026 : milvus.io, stackai.com, innovativeais.com, ailog.fr
- Comparatifs API LLM gratuites 2026 : openrouter.ai/blog, apiscout.dev, klymentiev.com, costbench.com

(Toutes les infos chiffrées — quotas, prix — évoluent vite : revérifie-les au moment de l'implémentation.)

18. MCP utiles pendant le développement (nano banana, etc.)

Les MCP (Model Context Protocol) sont des outils que tu branches à Claude (Desktop/Code) pendant que tu **développes** — ce ne sont pas des briques qui tournent dans l'app finale pour tes utilisateurs. Distinction importante à garder en tête.

MCP	Réalisable pour ce projet ?	Usage recommandé
Nano Banana (génération/édition d'images via Gemini, plusieurs serveurs MCP open source existent : <code>nanobanana-mcp</code> , <code>@nanana-ai/mcp-server-nano-banana</code> , etc.)	✅ Oui, très utile en phase de conception	Génère les icônes , les visuels de fond de la page, les mockups d'interface , une texture ou un skybox pour la scène 3D, des illustrations génériques (ex: pictogrammes bagages/douane). Ne l'utilise pas pour générer de fausses photos de plats/restaurants — ça nuit à la crédibilité d'une info qui doit être factuelle.
« hexfiled »	❓ Je ne trouve pas de MCP officiel sous ce nom précis (mes recherches renvoient HexDocs MCP, pour la doc de packages Elixir, ou HexStrike, pour la cybersécurité — aucun des deux ne correspond à ton besoin). Peux-tu me confirmer l'orthographe exacte ou un lien ? Si tu penses à un outil de génération/édition 3D , dis-le-moi, je regarde une alternative adaptée (ex: MCP Blender).	

Ce que ces MCP ne remplacent pas : ils ne font pas le pipeline RAG, ni le backend, ni la logique métier — ce sont des accélérateurs ponctuels pour la partie visuelle pendant que tu codes avec Claude Code.

19. Version application mobile

Trois options, du plus simple au plus complet :

Option	Effort	Recommandation
PWA (Progressive Web App) à partir du même code React web	Faible — tu ajoutes juste un manifest + service worker à l'appli existante	✅ Recommandé pour commencer : installable sur iOS/Android depuis le navigateur, 100 % gratuit, zéro code dupliqué, la scène 3D fonctionne telle quelle (c'est juste le navigateur)
React Native (via Expo)	Moyen — logique métier (appels API, chat, state) réutilisable à ~60-70 %, mais l'UI est à refaire en composants natifs	À envisager si tu veux être sur l'App Store / Play Store avec des notifications push natives (utile pour l'alerte de changement de porte, §14)
Flutter	Élevé — nouveau langage (Dart), rien de réutilisable de React	❌ Pas recommandé ici, tu perdrais tout le travail déjà fait en React

Point d'attention 3D sur mobile : `react-three-fiber` a un équivalent pour React Native (`@react-three/fiber` + `expo-gl`), mais les performances sur téléphone d'entrée de gamme sont plus limitées — prévois systématiquement le fallback léger (avion en SVG/CSS animé) mentionné en §9, en particulier pour la version mobile.

Recommandation finale : démarre avec la PWA (aucun travail supplémentaire, gratuit, rapide) ; migre vers **React Native/Expo** seulement si tu as un besoin réel de notifications push natives ou de présence sur les stores.

20. Restaurants et boutiques — modèle de données détaillé

Voici une structure JSON de départ que tu peux utiliser telle quelle pour `services.json` :

```
json
```

```

{
  "id": "resto_001",
  "type": "restaurant",
  "nom": "Nom du restaurant",
  "categorie": "café / fast-food / restaurant / boutique duty-free / presse / s",
  "zone": "Salle d'embarquement A",
  "porte_proche": "Porte 21",
  "etage": "1",
  "horaires": "05h00 - 23h00",
  "halal": true,
  "prix_moyen_mad": 60,
  "menu": [
    { "plat": "Tajine poulet", "prix_mad": 65, "categorie": "plat" },
    { "plat": "Café", "prix_mad": 15, "categorie": "boisson" }
  ],
  "photos": ["url_photo_reelle_1.jpg"],
  "derniere_mise_a_jour": "2026-08-01"
}

```

Points à respecter :

- `derniere_mise_a_jour` est essentiel : les menus/prix changent souvent — le chatbot doit pouvoir dire « ces prix datent du [date], à confirmer sur place » plutôt que d'affirmer une info potentiellement obsolète (cohérent avec la règle de fiabilité du §11).
- `porte_proche` et `zone` sont les champs qui alimentent le graphe de connaissances (§8) pour répondre à « qu'est-ce qui est ouvert près de ma porte ? ».
- Prévois un **cycle de mise à jour régulier** (ex: trimestriel) pour ces données, car contrairement aux vols, il n'existe pas d'API pour les menus d'un aéroport — c'est fatalement de la collecte manuelle.

21. Informations qu'il me manque pour affiner ce cahier des charges

- Cible principale : application mobile, site web, ou borne physique dans l'aéroport ?
- As-tu déjà un budget (même minime) pour de l'hébergement, ou faut-il rester 100% gratuit ?
- Le projet est-il académique (PFE/mémoire à soutenir) ou destiné à un usage réel avec de vrais utilisateurs ?
- As-tu un accès privilégié à l'ONDA (stage, partenariat) qui permettrait d'obtenir de vraies données, ou faut-il tout collecter publiquement ?
- Niveau technique de l'équipe : débutant, intermédiaire, ou expérimenté en Python/React ?